Structured Software Development

Ground to Mastery — Study Notes

Programming in C — Algorithms to File I/O & Sorting

CSC 122 | Year 1 Semester 2 | DCS

Coursework 40% (Test 10% + Assignment 10% + Practical 20%) | Final Exam 60%

How to Study This Course

This is your most practical course. Every topic must be coded in C and run. Read the concept, type the code in Neovim, compile with gcc, run, change values, break it, fix it. Exam questions always require writing and tracing C code.

> ⚡ *Workflow: tmux new -s csc122 → Ctrl+A | → left pane: nvim code.c → right pane: gcc -Wall code.c -o out && ./out*

> 📝 *Assessment: Test 10% + Assignment 10% + Practical 20% + Exam 60%. Every unit is examinable.*

UNIT 1: ALGORITHMS, FLOWCHARTS & PROGRAMME DESIGN

Designing the solution before writing code

▸ 1.1 — What is an Algorithm?

An algorithm is a finite, ordered set of well-defined instructions to solve a problem. Every C programme starts as an algorithm.

Term / Concept	Definition / Explanation
`Algorithm`	Step-by-step instructions to solve a problem — language independent — must be finite
`Pseudocode`	Algorithm written in structured plain English — closer to code than English
`Flowchart`	Visual diagram of an algorithm using standard shapes and arrows
`Programme`	Algorithm written in a programming language — in this course, C
`Bug`	Error in a programme — syntax error (compiler catches) or logic error (you find)

▸ 1.2 — Flowchart Symbols

Term / Concept	Definition / Explanation
`Oval / Terminal`	Start and End of programme
`Rectangle / Process`	Calculation or assignment: sum = a + b
`Diamond / Decision`	Condition — Yes/No branch — used for if, while, for
`Parallelogram / I-O`	Input (read) or Output (print)
`Arrow / Flowline`	Direction of flow between steps
`Circle / Connector`	Connects parts of flowchart across pages

▸ 1.3 — Programme Development Steps

1. Define the problem — what exactly is required?
2. Analyse — identify inputs, outputs, processes
3. Design — write pseudocode or draw flowchart
4. Code — write the C programme
5. Test — compile, run with various inputs
6. Debug — fix all syntax and logic errors
7. Document and Maintain — comment code, update as needed

> 📝 *Exam: Given a problem, write pseudocode THEN the C code. Always do pseudocode first — it organises your thinking and earns marks.*

▸ 1.4 — Structure of a C Programme

structure.c — compile: gcc -Wall structure.c -o out && ./out

```c
/* Complete structure of a C programme */

// 1. Preprocessor directives — include libraries
#include <stdio.h>       // printf, scanf, fopen, fclose
#include <stdlib.h>      // malloc, free, exit, atoi
#include <string.h>      // strlen, strcpy, strcmp, strcat
#include <math.h>        // sqrt, pow, sin — compile with -lm

// 2. Macros
#define MAX 100
#define PI  3.14159

// 3. Global variables
int globalCounter = 0;

// 4. Function prototypes
int add(int a, int b);

// 5. main() — programme starts here
int main() {
    int x = 10, y = 20;
    printf("Sum: %d\n", add(x, y));
    return 0;  // 0 = programme ran successfully
}

// 6. Function definitions
int add(int a, int b) {
    return a + b;
}
```

UNIT 2: TOKENS, DATA TYPES, VARIABLES & OPERATORS

The building blocks of every C programme

▸ 2.1 — C Tokens (6 Types)

Term / Concept	Definition / Explanation
`Keywords`	Reserved — int float char if while for return void struct — cannot use as names
`Identifiers`	Your names — variables, functions — start with letter or _ — case sensitive
`Constants`	Fixed values — 42 3.14 'A' "Hello"
`Operators`	Symbols — + - * / % == != && || etc.
`Strings`	Text in double quotes — "Hello World" — char array ending with \0
`Special symbols`	{ } ( ) [ ] ; , # — structural characters

▸ 2.2 — Identifier Rules

- Must start with a letter (a-z, A-Z) or underscore _
- Can contain letters, digits (0-9), underscores only — no spaces
- Cannot be a C keyword (int, float, if, for, while, etc.)
- Case sensitive: age, Age, AGE are three different identifiers

▸ 2.3 — All Data Types

Term / Concept	Definition / Explanation
`char`	1 byte — single character — 'A' '5' — range -128 to 127
`unsigned char`	1 byte — 0 to 255
`int`	4 bytes — whole numbers — -2,147,483,648 to 2,147,483,647 — use %d
`unsigned int`	4 bytes — 0 to 4,294,967,295 — use %u
`short int`	2 bytes — smaller range integers
`long int`	8 bytes — very large integers — use %ld
`float`	4 bytes — decimal — ~7 digits precision — use %f
`double`	8 bytes — precise decimal — ~15 digits — use %lf
`void`	No value — function returns nothing, or untyped pointer

▸ 2.4 — Variables, Constants & sizeof

variables.c

```c
#include <stdio.h>
#define MAX  100         // preprocessor constant — no type or ;
#define PI   3.14159

int main() {
    // Variable declarations
    int    age    = 25;
    float  height = 5.9f;
    double salary = 1500000.50;
    char   grade  = 'A';
    char   name[50] = "Bravehart";

    // const — cannot be changed after declaration
    const int DAYS = 7;
    // DAYS = 8;  // ERROR — const cannot be modified

    // sizeof — size in bytes
    printf("char=%zu  int=%zu  float=%zu  double=%zu\n",
           sizeof(char), sizeof(int), sizeof(float), sizeof(double));
    // Output: char=1  int=4  float=4  double=8

    printf("%s is %d years old, height %.1f\n", name, age, height);
    return 0;
}
```

▸ 2.5 — All Operators

Term / Concept	Definition / Explanation
`+ - * / %`	Arithmetic — add subtract multiply divide modulus (remainder)
`== != < > <= >=`	Relational — compare two values — return 1 (true) or 0 (false)
`&& || !`	Logical — AND OR NOT — combine conditions
`= += -= *= /= %=`	Assignment — assign and compound assign
`++ --`	Increment/decrement — prefix (++x: change first) postfix (x++: use first)
`(type)`	Cast — (float)a converts a to float type
`? :`	Ternary — (cond) ? value_if_true : value_if_false
`sizeof`	Size in bytes: sizeof(int) = 4 sizeof(arr)/sizeof(arr[0]) = array length
`& *`	Address-of (&x), Dereference (*ptr) — for pointers
`& | ^ ~ << >>`	Bitwise AND OR XOR NOT left-shift right-shift

operators.c

```c
#include <stdio.h>
int main() {
    int a=10, b=3;
```

```
    printf("%d %d %d %d %d\n", a+b,a-b,a*b,a/b,a%b); // 13 7 30 3 1

    // Integer division truncates decimal part
    printf("%d\n",   7/2);          // 3 (NOT 3.5)
    printf("%.2f\n", (float)7/2);   // 3.50 — cast first

    // Prefix vs postfix
    int x=5;
    printf("%d\n", ++x); // 6 — increment FIRST then use
    printf("%d\n", x++); // 6 — use FIRST then increment
    printf("%d\n", x);   // 7

    // Ternary
    int max = (a>b) ? a : b;
    printf("Max=%d\n", max); // 10
    return 0;
}
```

▸ 2.6 — Format Specifiers

Term / Concept	Definition / Explanation
`%d` `%i`	int
`%f`	float — %.2f prints 2 decimal places
`%lf`	double
`%c`	char — single character
`%s`	string — char array
`%p`	pointer address
`%ld`	long int
`%u`	unsigned int
`%zu`	size_t — use with sizeof
`%x`	hexadecimal
`\n` `\t` `\\` `\"`	newline tab backslash double-quote

UNIT 3: CONTROL FLOW — CONDITIONS & LOOPS

if/else, switch, while, for, do-while, break, continue, goto

▸ 3.1 — If / Else If / Else

if-else.c

```c
#include <stdio.h>
int main() {
    int score;
    printf("Enter score (0-100): ");
    scanf("%d", &score);

    if      (score >= 80) printf("Distinction\n");
    else if (score >= 70) printf("Credit\n");
    else if (score >= 60) printf("Pass\n");
    else if (score >= 50) printf("Bare Pass\n");
    else                  printf("Fail\n");

    // Multiple conditions with && and ||
    int age=20; int hasID=1;
    if (age>=18 && hasID)       printf("Access allowed\n");
    else if (age>=18 && !hasID) printf("Need ID\n");
    else                        printf("Too young\n");
    return 0;
}
```

▸ 3.2 — Switch Statement

switch.c

```c
#include <stdio.h>
int main() {
    char op; double a, b;
    printf("Enter: num op num: ");
    scanf("%lf %c %lf", &a, &op, &b);

    switch (op) {
        case '+': printf("= %.2f\n", a+b); break;
        case '-': printf("= %.2f\n", a-b); break;
        case '*': printf("= %.2f\n", a*b); break;
        case '/':
            if (b != 0) printf("= %.2f\n", a/b);
            else printf("Error: division by zero\n");
            break;
        default: printf("Unknown operator '%c'\n", op);
    }
    // switch only works with int and char — NOT float/string
    // break is REQUIRED in each case — without it, falls through
    return 0;
}
```

▸ 3.3 — All Loop Types

loops.c

```c
#include <stdio.h>
int main() {
    // FOR — when you know the count
    int sum = 0;
    for (int i=1; i<=100; i++) sum += i;
    printf("Sum 1 to 100 = %d\n", sum); // 5050

    // WHILE — unknown iterations
    int n=12345, digits=0;
    while (n > 0) { digits++; n /= 10; }
    printf("Digits: %d\n", digits); // 5

    // DO-WHILE — runs at least once — good for menus
    int choice;
    do {
        printf("1.Add  2.Sub  3.Exit: ");
        scanf("%d", &choice);
    } while (choice < 1 || choice > 3);

    // NESTED loops — star pattern
    for (int i=1; i<=5; i++) {
        for (int j=1; j<=i; j++) printf("* ");
        printf("\n");
    }

    // break exits loop, continue skips to next iteration
    for (int i=0; i<=20; i++) {
        if (i==15) break;       // stop at 15
        if (i%2==0) continue;   // skip even numbers
        printf("%d ", i);       // 1 3 5 7 9 11 13
    }
    printf("\n");
    return 0;
}
```

▸ 3.4 — goto and Labels

goto.c

```c
#include <stdio.h>
int main() {
    int i=1;
loop:
    if (i>5) goto done;
    printf("%d\n", i);
    i++;
    goto loop;
done:
    printf("Done\n");
    return 0;
```

```
}
```

> ⚠️ *goto is considered bad practice in structured programming — it makes code hard to follow and maintain. Know it for the exam but avoid it in practice.*

UNIT 4: FUNCTIONS & STORAGE CLASSES

Structured programme design — functions, parameters, scope

▸ 4.1 — Functions

functions.c — compile: gcc -Wall functions.c -lm -o out && ./out

```
#include <stdio.h>
#include <math.h>

// Prototypes — must appear BEFORE main
int    isPrime(int n);
double circleArea(double r);
int    fibonacci(int n);
void   swap(int *a, int *b);

int main() {
    // Print primes 2 to 50
    for (int i=2; i<=50; i++)
        if (isPrime(i)) printf("%d ", i);
    printf("\n");

    printf("Area(r=5): %.2f\n", circleArea(5));

    int a=10, b=20;
    swap(&a, &b);
    printf("%d %d\n", a, b); // 20 10
    return 0;
}

int isPrime(int n) {
    if (n < 2) return 0;
    for (int i=2; i*i<=n; i++)
        if (n%i==0) return 0;
    return 1;
}
double circleArea(double r) { return 3.14159 * r * r; }
int fibonacci(int n) {
    if (n<=0) return 0;
    if (n==1) return 1;
    return fibonacci(n-1) + fibonacci(n-2); // recursive
}
void swap(int *a, int *b) {  // pass by reference
    int t=*a; *a=*b; *b=t;
}
```

▸ 4.2 — Pass by Value vs Pass by Reference

Term / Concept	Definition / Explanation
`Pass by value`	A COPY of the variable is passed — original NOT changed — default in C
`Pass by reference`	The ADDRESS (&var) is passed — function can change the

	original — use * in function
`Arrays`	Arrays are ALWAYS passed by reference — address of first element is passed

▸ 4.3 — Storage Classes

Term / Concept	Definition / Explanation
`auto`	Default for local variables — exists only during function execution — re-created each call
`static (local)`	Local variable that KEEPS its value between calls — initialized only once
`static (global)`	Global visible only in current file — prevents access from other files
`extern`	Variable defined in another file — used for sharing global variables across files
`register`	Hint to use CPU register — faster access — cannot take address with &

storage-classes.c

```c
#include <stdio.h>
void counter() {
    auto   int a = 0;  // reset to 0 every call
    static int s = 0;  // keeps value between calls
    a++; s++;
    printf("auto=%d  static=%d\n", a, s);
}
int main() {
    counter(); // auto=1  static=1
    counter(); // auto=1  static=2
    counter(); // auto=1  static=3
    return 0;
}
```

▸ 4.4 — Recursive Functions

recursion.c

```c
#include <stdio.h>
// Factorial — n! = n*(n-1)*(n-2)*...*1
long factorial(int n) {
    if (n<=1) return 1;          // base case
    return n * factorial(n-1);   // recursive call
}
// Power — base^exp
double power(double base, int exp) {
    if (exp==0) return 1;
    return base * power(base, exp-1);
}
```

```
// Sum of digits
int sumDigits(int n) {
    if (n==0) return 0;
    return (n%10) + sumDigits(n/10);
}
int main() {
    printf("5! = %ld\n", factorial(5));       // 120
    printf("2^10 = %.0f\n", power(2,10));     // 1024
    printf("sumDigits(12345) = %d\n", sumDigits(12345)); // 15
    return 0;
}
```

▸ 4.5 — Standard Library Functions

Term / Concept	Definition / Explanation
`<stdio.h>`	printf scanf fgets fopen fclose fprintf fscanf fread fwrite feof rewind
`<stdlib.h>`	malloc calloc realloc free atoi atof rand srand exit abort
`<string.h>`	strlen strcpy strncpy strcat strcmp strstr strchr strtok
`<math.h>`	sqrt pow fabs ceil floor sin cos tan — add -lm when compiling
`<ctype.h>`	isalpha isdigit isspace toupper tolower isupper islower
`<time.h>`	time clock difftime — used for timing and seeding rand()

UNIT 5: ARRAYS & POINTERS

Most exam-heavy unit — understand every line

▸ 5.1 — One-Dimensional Arrays

arrays1d.c

```c
#include <stdio.h>
#define N 6
int main() {
    int arr[N] = {64,25,12,22,11,45};

    // Find sum, min, max
    int sum=0, min=arr[0], max=arr[0];
    for (int i=0; i<N; i++) {
        sum += arr[i];
        if (arr[i]<min) min=arr[i];
        if (arr[i]>max) max=arr[i];
    }
    printf("Sum=%d Min=%d Max=%d Avg=%.2f\n", sum,min,max,(float)sum/N);

    // Search for a value
    int target=22, found=-1;
    for (int i=0; i<N; i++)
        if (arr[i]==target) { found=i; break; }
    if (found>=0) printf("Found %d at index %d\n", target, found);
    else printf("%d not found\n", target);

    // Array size trick
    int size = sizeof(arr)/sizeof(arr[0]);
    printf("Array has %d elements\n", size); // 6
    return 0;
}
```

▸ 5.2 — Two-Dimensional Arrays (Matrices)

arrays2d.c

```c
#include <stdio.h>
#define R 3
#define C 3
void print(int m[R][C]) {
    for (int i=0;i<R;i++){
        for(int j=0;j<C;j++) printf("%4d",m[i][j]);
        printf("\n");
    }
}
int main() {
    int A[R][C]={{1,2,3},{4,5,6},{7,8,9}};
    int B[R][C]={{9,8,7},{6,5,4},{3,2,1}};
    int S[R][C];
    // Add matrices
    for(int i=0;i<R;i++)
```

```
        for(int j=0;j<C;j++)
            S[i][j]=A[i][j]+B[i][j];
    printf("A:\n"); print(A);
    printf("B:\n"); print(B);
    printf("A+B:\n"); print(S);
    // Diagonal sum
    int diag=0;
    for(int i=0;i<R;i++) diag+=A[i][i];
    printf("Diagonal sum: %d\n",diag); // 1+5+9=15
    return 0;
}
```

▸ 5.3 — Strings (char arrays)

strings.c

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main() {
    char s1[50]="Hello", s2[50]="World", s3[100];

    printf("Length: %zu\n", strlen(s1));        // 5
    strcpy(s3, s1);                              // copy s1 to s3
    strcat(s3, " ");  strcat(s3, s2);            // s3 = "Hello World"
    printf("%s\n", s3);
    printf("Compare: %d\n", strcmp(s1,s2));     // negative (H < W)
    printf("Substring: %s\n", strstr(s3,"World")); // "World"

    // Convert to uppercase
    for (int i=0; s1[i]; i++) s1[i]=toupper(s1[i]);
    printf("%s\n", s1); // HELLO

    // Count vowels
    char str[]="Bravehart"; int vowels=0;
    for(int i=0;str[i];i++)
        if(strchr("aeiouAEIOU",str[i])) vowels++;
    printf("Vowels in %s: %d\n",str,vowels); // 3
    return 0;
}
```

▸ 5.4 — Pointers — Complete

pointers.c

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    // Pointer basics
    int  x   = 42;
    int *ptr = &x;  // & = address-of
```

```
    printf("x=%d  &x=%p  *ptr=%d\n", x, ptr, *ptr);
    *ptr = 99;       // change x through pointer
    printf("x is now: %d\n", x); // 99

    // Array and pointer equivalence
    int arr[]={10,20,30,40,50};
    int *p=arr;
    printf("%d %d %d\n", *p, *(p+1), p[2]); // 10 20 30

    // Pass array to function — modifies original
    // void doubleAll(int *a, int n) — use a[i]*=2

    // Dynamic memory
    int n=5;
    int *dyn=(int*)malloc(n*sizeof(int));
    if (!dyn) { printf("malloc failed\n"); return 1; }
    for(int i=0;i<n;i++) dyn[i]=(i+1)*10;
    for(int i=0;i<n;i++) printf("%d ",dyn[i]); // 10 20 30 40 50
    printf("\n");
    free(dyn); dyn=NULL;  // ALWAYS free, ALWAYS set to NULL
    return 0;
}
```

Term / Concept	Definition / Explanation
`& (address-of)`	Gets the memory address of a variable: &x returns 0x7fff...
`* (dereference)`	Gets the value at a pointer address: *ptr returns the value stored there
`NULL pointer`	A pointer not pointing to anything — always initialize: int *p = NULL
`malloc(n)`	Allocates n bytes — returns void* — must cast: (int*)malloc(...)
`calloc(n, size)`	Allocates n*size bytes and zeros them all
`realloc(ptr, n)`	Resize previously allocated memory
`free(ptr)`	ALWAYS free dynamic memory when done — otherwise memory leak

> 📝 *Exam: Write a programme to reverse an array using pointers. Use two pointers — start and end — swap and move inward.*

UNIT 6: STRUCTURES & UNIONS

User-defined types for grouping related data

▸ 6.1 — Structures

structures.c

```c
#include <stdio.h>
#include <string.h>
typedef struct {
    int   regNo;
    char  name[50];
    float marks[3];
    float average;
    char  grade;
} Student;

char getGrade(float avg) {
    if(avg>=80) return 'D';
    if(avg>=70) return 'C';
    if(avg>=60) return 'P';
    return 'F';
}
void process(Student *s) {
    float sum=0;
    for(int i=0;i<3;i++) sum+=s->marks[i];
    s->average=sum/3;
    s->grade=getGrade(s->average);
}
void print(const Student *s) {
    printf("[%d] %-20s Avg:%.2f Grade:%c\n",
           s->regNo,s->name,s->average,s->grade);
}
int main() {
    Student class[3]={
        {2401,"Bravehart",{75,82,90}},
        {2402,"Alice",    {88,91,85}},
        {2403,"Bob",      {55,60,58}},
    };
    for(int i=0;i<3;i++){process(&class[i]);print(&class[i]);}
    return 0;
}
```

▸ 6.2 — Unions

unions.c

```c
#include <stdio.h>
union Data { int i; float f; char str[20]; };
int main() {
    union Data d;
    printf("Union size: %zu\n", sizeof(d)); // 20 — largest member
    d.i = 65;
```

```
    printf("i=%d  char=%c\n", d.i, d.str[0]); // 65  A
    d.f = 3.14;  // overwrites d.i !
    printf("f=%.2f  i=%d (garbage)\n", d.f, d.i);
    return 0;
}
```

Term / Concept	Definition / Explanation
`struct size`	Sum of all member sizes (plus alignment padding)
`union size`	Size of the LARGEST member only — all share same memory
`dot (.) operator`	Access member by value: student.name student.age
`arrow (->) operator`	Access member through pointer: ptr->name ptr->age
`typedef struct`	Creates a type alias so you can write Student instead of struct Student
`Nested struct`	A struct containing another struct as a member

UNIT 7: FILE INPUT & OUTPUT

Text files, binary files, streams, error handling

fileio.c

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct { int id; char name[30]; float mark; } Record;

int main() {
    // TEXT WRITE
    FILE *fw=fopen("students.txt","w");
    if(!fw){perror("fopen");exit(1);}
    fprintf(fw,"%d %s %.2f\n",1,"Bravehart",88.5);
    fprintf(fw,"%d %s %.2f\n",2,"Alice",92.0);
    fclose(fw);

    // TEXT READ
    FILE *fr=fopen("students.txt","r");
    Record r;
    while(fscanf(fr,"%d %s %f",&r.id,r.name,&r.mark)==3)
        printf("ID:%-3d Name:%-15s Mark:%.2f\n",r.id,r.name,r.mark);
    fclose(fr);

    // BINARY WRITE
    Record rec={1,"Bravehart",88.5};
    FILE *fb=fopen("data.bin","wb");
    fwrite(&rec,sizeof(Record),1,fb);
    fclose(fb);

    // BINARY READ
    Record rec2;
    fb=fopen("data.bin","rb");
    fread(&rec2,sizeof(Record),1,fb);
    printf("Read: %d %s %.2f\n",rec2.id,rec2.name,rec2.mark);
    fclose(fb);
    return 0;
}
```

Term / Concept	Definition / Explanation
`fopen(f,"r")`	Open for reading — file must exist
`fopen(f,"w")`	Open for writing — creates or overwrites file
`fopen(f,"a")`	Open to append — adds to end of existing file
`fopen(f,"rb")`	Open binary for reading
`fopen(f,"wb")`	Open binary for writing
`fclose(fp)`	ALWAYS close file after use — flushes buffers
`fprintf(fp,...)`	Write formatted text to file (like printf but to file)
`fscanf(fp,...)`	Read formatted text from file (like scanf but from file)
`fgets(buf,n,fp)`	Read one line of text from file safely

`fwrite(&s,sz,n,fp)`	Write n binary records of size sz
`fread(&s,sz,n,fp)`	Read n binary records of size sz
`feof(fp)`	Returns true when end of file reached
`rewind(fp)`	Move file pointer back to beginning
`fseek(fp,off,origin)`	Move to specific position — SEEK_SET SEEK_CUR SEEK_END
`perror("msg")`	Print system error message — use after fopen returns NULL

UNIT 8: SORTING ALGORITHMS

Bubble, Insertion, Quick Sort — code + trace

▸ 8.1 — Bubble Sort — O(n²)

Compare adjacent elements — swap if out of order. Largest element bubbles to end each pass.

bubble-sort.c

```c
#include <stdio.h>
void bubbleSort(int a[], int n) {
    for (int i=0; i<n-1; i++) {
        int swapped=0;
        for (int j=0; j<n-i-1; j++) {
            if (a[j]>a[j+1]) {
                int t=a[j]; a[j]=a[j+1]; a[j+1]=t;
                swapped=1;
            }
        }
        if (!swapped) break; // already sorted — early exit
    }
}
int main() {
    int a[]={64,25,12,22,11}; int n=5;
    bubbleSort(a,n);
    for(int i=0;i<n;i++) printf("%d ",a[i]); // 11 12 22 25 64
    printf("\n");
}
```

▸ 8.2 — Insertion Sort — O(n²) worst, O(n) best

Build sorted portion one element at a time. Pick next element, shift sorted elements right until correct position found, insert.

insertion-sort.c

```c
#include <stdio.h>
void insertionSort(int a[], int n) {
    for (int i=1; i<n; i++) {
        int key=a[i], j=i-1;
        while (j>=0 && a[j]>key) {
            a[j+1]=a[j]; j--;
        }
        a[j+1]=key;
    }
}
int main() {
    int a[]={12,11,13,5,6}; int n=5;
    insertionSort(a,n);
    for(int i=0;i<n;i++) printf("%d ",a[i]); // 5 6 11 12 13
}
```

▸ 8.3 — Quick Sort — O(n log n) average

Choose pivot. Partition: smaller go left, larger go right. Recursively sort both halves.

quick-sort.c

```c
#include <stdio.h>
void swap(int*a,int*b){int t=*a;*a=*b;*b=t;}

int partition(int a[],int low,int high){
    int pivot=a[high],i=low-1;
    for(int j=low;j<high;j++)
        if(a[j]<=pivot){i++;swap(&a[i],&a[j]);}
    swap(&a[i+1],&a[high]);
    return i+1;
}
void quickSort(int a[],int low,int high){
    if(low<high){
        int pi=partition(a,low,high);
        quickSort(a,low,pi-1);
        quickSort(a,pi+1,high);
    }
}
int main(){
    int a[]={10,7,8,9,1,5}; int n=6;
    quickSort(a,0,n-1);
    for(int i=0;i<n;i++) printf("%d ",a[i]); // 1 5 7 8 9 10
}
```

Term / Concept	Definition / Explanation
`Bubble Sort`	O(n^2) — Simple to code and trace — compare adjacent, swap — best for exam traces
`Insertion Sort`	O(n^2) worst, O(n) best — efficient for small/nearly sorted arrays — stable
`Quick Sort`	O(n log n) average — fastest in practice — divide and conquer — unstable
`Stable sort`	Equal elements keep their original relative order after sorting

> 📝 *EXAM: Trace bubble sort on {5,3,8,1,2} — draw a table showing each pass, each comparison and every swap. Know how many passes are needed.*

Neovim + Tmux Workflow for CSC 122

CSC122 Neovim + Tmux workflow

```
# Create practice directory structure
mkdir -p ~/csc122/{u1-algorithms,u2-datatypes,u3-loops,u4-functions,u5-
arrays,u6-structs,u7-files,u8-sorting}
cd ~/csc122

# Start named tmux session
tmux new -s csc122

# Split: left=editor, right=terminal
Ctrl+A → |

# LEFT pane — open Neovim
nvim u1-algorithms/hello.c

# RIGHT pane — compile and run
gcc -Wall u1-algorithms/hello.c -o /tmp/out && /tmp/out

# Run from inside Neovim (saves switching)
:!gcc -Wall % -o /tmp/out && /tmp/out

# For math functions add -lm:
:!gcc -Wall % -lm -o /tmp/out && /tmp/out

# Create new tmux window per unit
Ctrl+A → c       # new window
Ctrl+A → ,       # rename window: u2-datatypes
Ctrl+A → 1-8     # jump to window by number

# Zoom a pane to fullscreen when reading output
Ctrl+A → z    # zoom/unzoom
```

> ✅ *You now have complete notes for every unit of CSC 122. Type every code example in Neovim. The exam asks you to write C code — muscle memory from typing beats passive reading every time.*